

Memory Management

Heap vs stack; garbage collection basics; common memory pitfalls

Rahul Guha || SME || Java FS Training || HCL

Java Stack vs Heap Memory Allocation

In Java, memory allocation is primarily divided into two categories, i.e., Stack and Heap memory. Both are used for different purposes, and they have different characteristics.

- **Stack Memory:** Stores primitive local variables, method call information, and references to objects during program execution.
- **Heap Memory:** Stores actual objects and dynamic data allocated at runtime. Objects created with `new` are placed here, and this memory is managed by the Garbage Collector.
- Java handles memory management through two key areas: the stack and the heap. Understanding the differences between these two memory areas and how Java manages them via garbage collection is essential for writing efficient applications.

1. Stack Memory

The stack is a region of memory that stores local variables and method call information. It follows the Last In, First Out (LIFO) principle, where memory is allocated and freed in the reverse order in which it was used.

Key Features of Stack:

Used for: Primitive data types (int, char, boolean, etc.) || References to objects (but not the objects themselves) || Method call frames (including method parameters and local variables)

- **Memory Size:** Typically smaller and more limited than heap memory.
- **Memory Allocation:** Happens automatically as methods are called.
- **Memory Deallocation:** Happens when methods return, and the variables in those methods are automatically removed from the stack.
- **Speed:** Very fast compared to heap memory because access is predictable (LIFO).
- **Scope:** Variables stored in the stack are limited to the current method. Once the method finishes, the memory is cleared.
- **Thread-Specific:** Each thread has its own stack memory.

2. Heap Memory

The heap is a region of memory used to store objects and dynamic memory. It is larger and more flexible than the stack, but access to heap memory is slower because it doesn't follow the strict LIFO order.

Key Features of Heap:

Used for: Objects created using new (e.g., new Integer(5)) || Instance variables of objects

- **Memory Size:** Generally larger than the stack.
- **Memory Allocation:** Objects are dynamically allocated at runtime.
- **Memory Deallocation:** Managed by Java's Garbage Collector (more on this later), which automatically frees up memory that is no longer in use.
- **Speed:** Slower than stack memory, as objects are scattered in memory, and access requires pointer dereferencing.
- **Scope:** Objects in the heap persist as long as they are referenced by any part of the program. They are only deallocated when they are no longer referenced.

3. Stack vs. Heap: Key Differences

Stack	Heap
Stores local variables and method call frames	Stores objects and instance variables
Memory deallocation happens automatically when a method completes	Memory deallocation is handled by Garbage Collection
LIFO (Last In, First Out) memory management	Objects are randomly allocated and may be scattered
Smaller size , typically limited	Larger size , shared by the entire application
Faster access , due to its structure	Slower access , due to dynamic allocation
Thread-specific : Each thread has its own stack	Global : All threads share the heap memory

4. Garbage Collection

Java provides automatic memory management via **Garbage Collection (GC)**. The garbage collector automatically reclaims memory that is no longer reachable by the program, freeing developers from manual memory management (as in languages like C or C++).

How Garbage Collection Works:

- **Objects in the heap** that are no longer referenced by any part of the program are considered **garbage**.
- **GC Roots:** The garbage collector starts from "roots," which are references from active stack frames, static variables, or thread references, and tracks all objects that are still reachable.
- **Unreachable objects:** Any object that cannot be reached from GC roots is considered eligible for garbage collection.
- **Deallocation:** The garbage collector automatically deallocates the memory used by unreachable objects, freeing up heap space.

Phases of Garbage Collection:

1. **Marking:** The GC identifies all live (reachable) objects.
2. **Deletion (Sweeping):** It removes objects that are no longer reachable.
3. **Compacting (optional):** In some garbage collectors, the heap is compacted to eliminate fragmentation, making memory allocation more efficient.

Garbage Collection Types

There are different algorithms and approaches for garbage collection:

- Serial Garbage Collector: Single-threaded garbage collection; ideal for small applications.
- Parallel Garbage Collector: Uses multiple threads for garbage collection.
- CMS (Concurrent Mark-Sweep) Garbage Collector: Minimizes stop-the-world events by running alongside the application.
- G1 Garbage Collector: Divides the heap into regions and uses both parallel and concurrent approaches for efficiency.

Important Points to Remember:

The **stack** is used for method calls and local variables, while the **heap** is used for objects.

Stack memory is faster but limited, while heap memory is larger but slower.

Java's **Garbage Collector** automatically manages heap memory, reclaiming space from objects that are no longer in use.

Garbage Collection Types

Java provides automatic memory management via **Garbage Collection (GC)**. The garbage collector automatically reclaims memory that is no longer reachable by the program, freeing developers from manual memory management (as in languages like C or C++).

How Garbage Collection Works:

- **Objects in the heap** that are no longer referenced by any part of the program are considered **garbage**.
- **GC Roots:** The garbage collector starts from "roots," which are references from active stack frames, static variables, or thread references, and tracks all objects that are still reachable.
- **Unreachable objects:** Any object that cannot be reached from GC roots is considered eligible for garbage collection.
- **Deallocation:** The garbage collector automatically deallocates the memory used by unreachable objects, freeing up heap space.

Phases of Garbage Collection:

1. **Marking:** The GC identifies all live (reachable) objects.
2. **Deletion (Sweeping):** It removes objects that are no longer reachable.
3. **Compacting (optional):** In some garbage collectors, the heap is compacted to eliminate fragmentation, making memory allocation more efficient.

Comparable vs Comparator

Feature	<code>compareTo()</code>	Comparable	Comparator
What is it?	A method	An interface	An interface
Package	<code>java.lang</code>	<code>java.lang</code>	<code>java.util</code>
Main method	<code>compareTo()</code>	<code>compareTo()</code>	<code>compare()</code>
Purpose	Performs comparison	Defines natural/default ordering	Defines custom ordering
Where implemented?	Inside the class	The class being sorted	Usually a separate class/lambda
Modifies original class?	—	Yes	No
Number of sorting rules	Usually one natural order	Usually one natural order	Can have multiple
Example	<code>s1.compareTo(s2)</code>	<code>class Student implements Comparable<Student></code>	<code>Comparator<Student></code>
Used with	<code>Collections.sort()</code> / <code>Arrays.sort()</code>	<code>Collections.sort()</code> / <code>Arrays.sort()</code>	<code>Collections.sort()</code> / <code>List.sort()</code>
Best use	Compare two objects according to natural order	Define object's default sorting	Different/custom sorting requirements

Getters and Setters in Java

In Java, Getter and Setter are methods used to protect your data and make your code more secure. Getter and Setter make the programmer convenient in setting and getting the value for a particular data type.

Getter in Java: Getter returns the value (accessors), it returns the value of data type int, String, double, float, etc. For the program's convenience, the getter starts with the word "get" followed by the variable name.

Setter in Java: While Setter sets or updates the value (mutators). It sets the value for any variable used in a class's programs. and starts with the word "set" followed by the variable name.

AI Integration in Java

Using Copilot for Boilerplate Generation & Critically Reviewing AI-Generated OOP Code:

1. Introduction to AI-Assisted Programming AI-assisted programming means using AI coding tools such as GitHub Copilot to help developers write, explain, refactor, test, and document code.

For Java programming, AI tools can be particularly useful for generating repetitive code such as:

POJOs

Constructors

Getters and setters

toString()

equals() and hashCode

()Collection operations

Loops and iteration

Interfaces and basic implementations

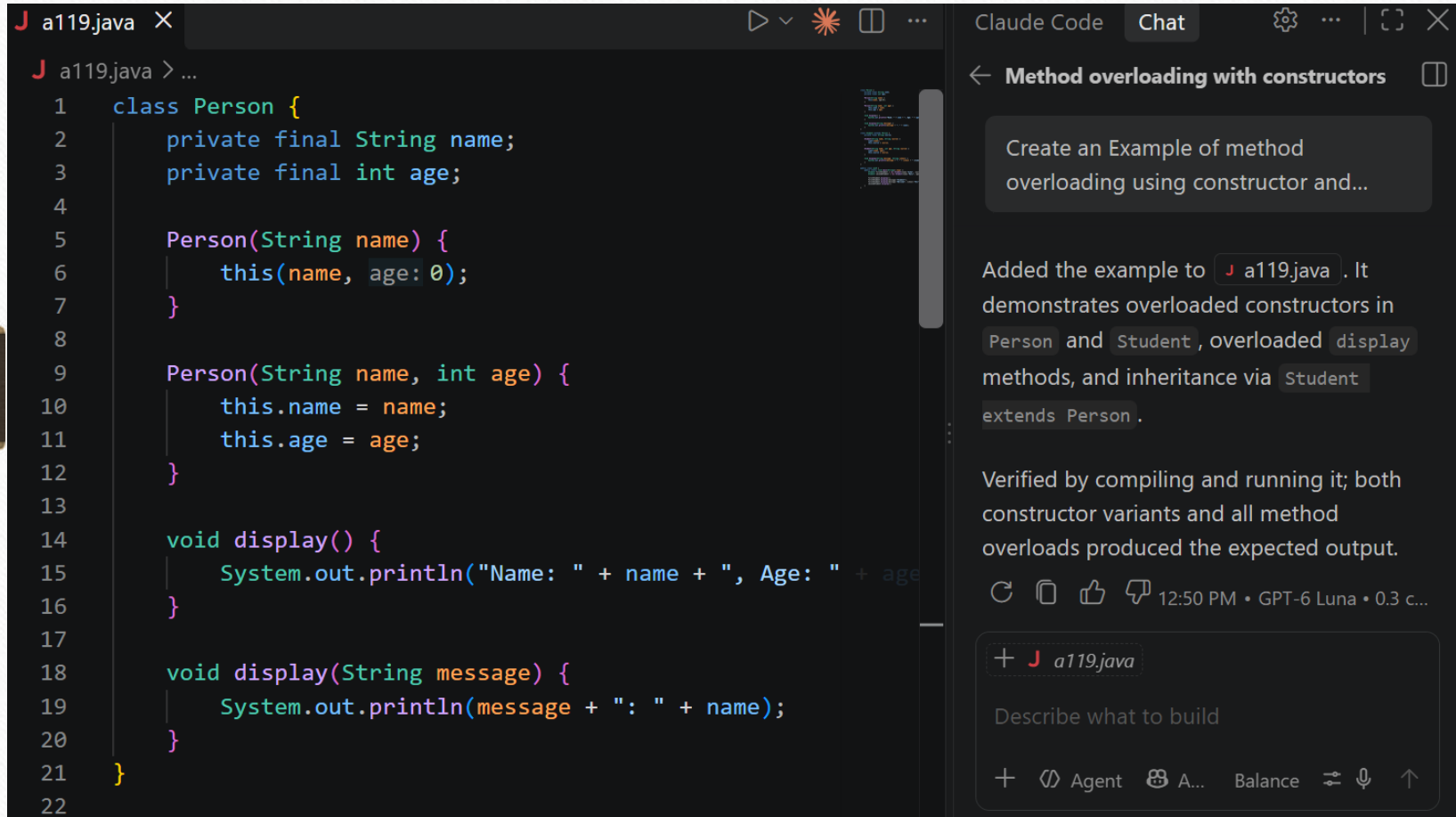
Unit-test templates

Documentation/comments

Basic workflow

```
graph TD; A[Problem / Requirement] --> B[Write prompt or partial code]; B --> C[AI generates code]; C --> D[Developer reviews code]; D --> E[Compile & Test]; E --> F[Check OOP Design]; F --> G[Security / Performance Review]; G --> H[Accept / Modify / Reject];
```

AI Integration in Java



The screenshot displays a code editor with a file named `a119.java`. The code defines a `Person` class with two constructors and two `display` methods. The first constructor takes a `String` name and initializes `age` to 0. The second constructor takes both a `String` name and an `int` age. The first `display` method prints the name and age, while the second `display` method prints a custom message along with the name.

```
1 class Person {
2     private final String name;
3     private final int age;
4
5     Person(String name) {
6         this(name, age: 0);
7     }
8
9     Person(String name, int age) {
10        this.name = name;
11        this.age = age;
12    }
13
14    void display() {
15        System.out.println("Name: " + name + ", Age: " + age);
16    }
17
18    void display(String message) {
19        System.out.println(message + ": " + name);
20    }
21 }
22
```

On the right side of the editor, there is a chat window titled "Claude Code Chat". The chat history shows a request to "Create an Example of method overloading using constructor and..." and a response explaining that the example was added to `a119.java`, demonstrating overloaded constructors, overloaded `display` methods, and inheritance via `Student` extending `Person`. The chat also includes a verification statement and a timestamp of 12:50 PM.

Step 1: Read Java File.

Step2: Run PWSH command – Compiles and runs the program.

Step 3: Edited

Step 4: Javac Rady Code

AI Integration in Java

2. What is GitHub Copilot?

GitHub Copilot is an AI-powered programming assistant that can generate code suggestions from natural-language instructions and surrounding code context.

It can help with:
Natural Language



AI understands intent



Code suggestion



Developer review



Final implementation

```
// Create a Student class with id, name and marks.
```

```
// Include constructor, getters, setters and toString().
```

3. What is Boilerplate Code?

- **Boilerplate code** is repetitive code that follows a standard pattern and does not contain much unique business logic.

For example:

- `private int id;`
- `private String name;`
- `private double marks;`

Followed By:

```
public int getId() {  
    return id;  
}  
  
public void setId(int id) {  
    this.id = id;  
}
```

AI Integration in Java

4. POJO

- **POJO = Plain Old Java Object**
- A POJO is generally a simple Java class used to represent data.
- **1. Generating POJOs and DTOs :** Instead of manually typing fields and generating methods via your IDE menu, you can use descriptive comments or partial class definitions to let Copilot write the entire structure.

Method A: Prompting with Comments

Type a multi-line comment describing your requirements. Copilot will suggest the fields, constructors, getters, setters, toString(), equals(), and hashCode() methods.

As you press **Enter** or **Tab**, Copilot will progressively generate the requested code block.

Example:

```
// Create a POJO named Product with fields: id (Long), name (String), price (BigDecimal), and tags (List of Strings).
```

- ```
// Include a no-arg constructor, all-args constructor, getters, setters, and toString.
```

# AI Integration in Java

## Method B: Starting the Class Definition

Copilot responds well to context. Just typing the fields and the beginning of a constructor will prompt it to complete the rest.

Example:

```
public class UserDto {
 private Long id;
 private String email;
 private boolean isActive; // Press Enter here, and Copilot will suggest:
 // public UserDto() {}
 // public UserDto(Long id, String email, boolean isActive) { ... }
 // Getters and Setters...}
```

# AI Integration in Java

**Method C:** Leveraging Modern Java (Records) If you are using Java 14 or higher, you can ask Copilot to use record syntax to drastically reduce boilerplate natively.

```
java// Create a Java record for an Order item with id, SKU, and quantitypublic record OrderItem(Long id, String sku, int quantity) {}
```

## 2. Streamlining Collections Code

- Copilot shines when writing logic for filtering, transforming, and initializing Java collections, especially when utilizing the **Streams API**.
- **Common Collection Tasks and Prompts**  
Filtering and Transforming:
  - `// Filter a list of Users to get only active users and collect their emails into a List<String>`  
`List<String> activeEmails = users.stream().filter(User::isActive).map(User::getEmail).collect(Collectors.toList());`